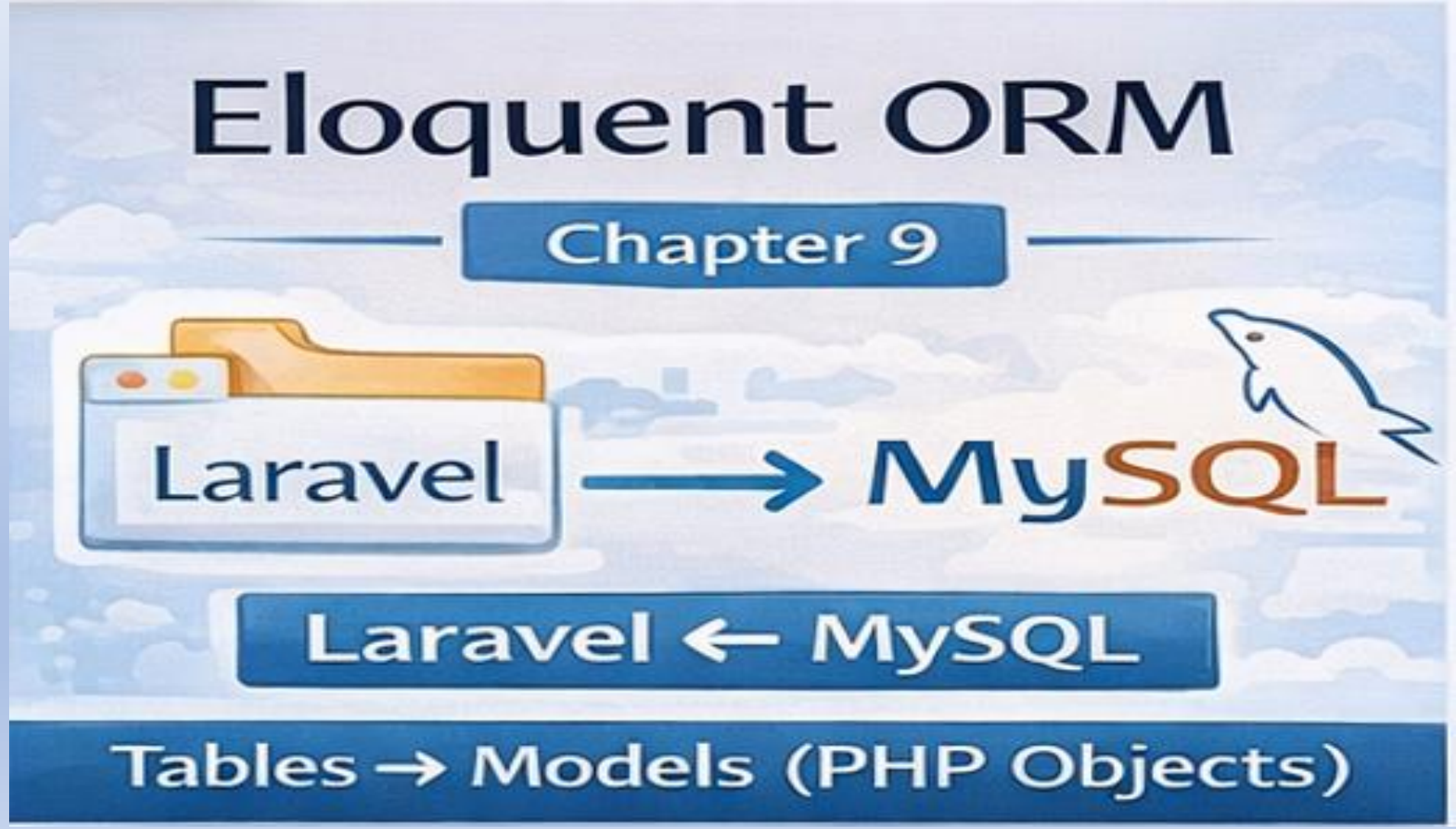
Chapter 9: Eloquent ORM



1. Objectives

1. Understand Eloquent ORM
2. Create and use models
3. Perform CRUD operations
4. Work with relationships

Explanation

Eloquent ORM is Laravel's **built-in system for interacting with databases using PHP objects instead of raw SQL**.

2. What is ORM?

ORM (Object Relational Mapping)

ORM connects:

Object (PHP Class) ↔ Database Table

Example

Object	Database
Student Model	students table

Explanation

ORM allows developers to work with **database records as objects**.

3. What is Eloquent?

Eloquent ORM

Eloquent is Laravel's **active record implementation**.

Example:

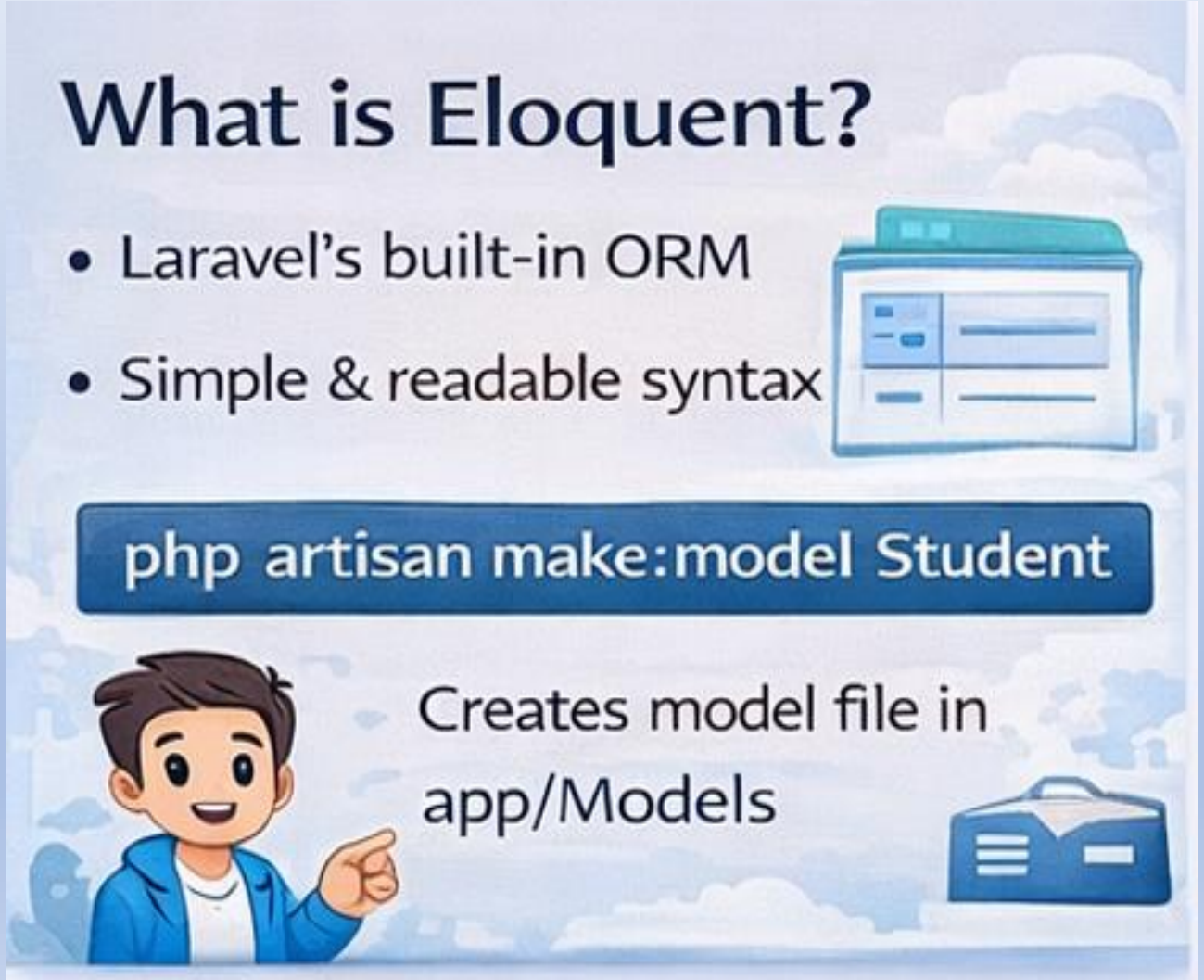
```php
Student::all();
```

Instead of SQL:

```sql
SELECT * FROM students;
```

Explanation

Eloquent simplifies **database queries and improves code readability**.



4 — Eloquent ORM Architecture

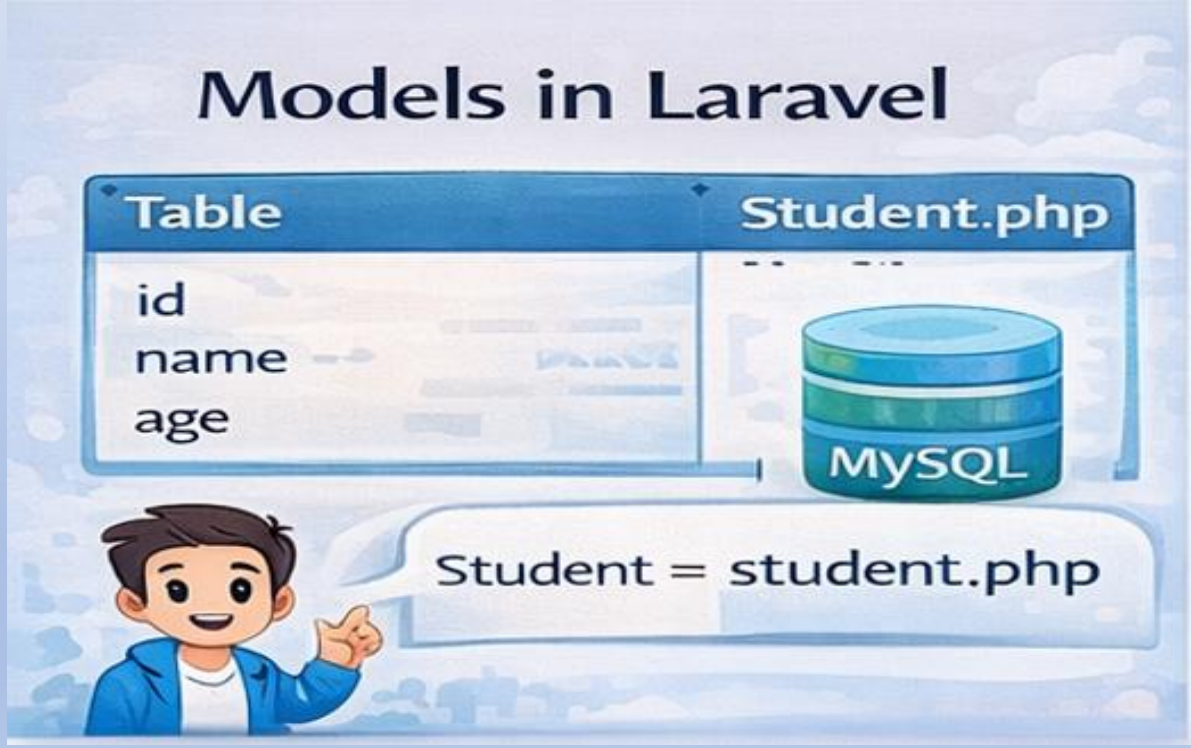
```
Model
  ↓
Eloquent ORM
  ↓
Database (MySQL)
```

Explanation

The **Model acts as the bridge between the application and database.**

What is a Model?

1. Represents a database table
2. One model = one table



Creating a Model

```bash
php artisan make:model Student
```

- Creates model in `app/Models`

Model Naming Rules

- Model: `Student`
- Table: `students`
- Automatic mapping

Basic Model Example

Path

```
app/Models/Student.php
```

```php
class Student extends Model {
}
```

Explanation

`$fillable` protects against **mass assignment vulnerability**.

7 — Creating Model with Migration

Command:

```bash
php artisan make:model Student -m
```

This creates:

```
Model
Migration
```

Explanation

Migration defines the **structure of database tables**.

8 — Migration Example

Path

```
database/migrations/create_students_table.php
```

Code

PHP

```php
Schema::create('students', function (Blueprint $table) {
$table->id();
$table->string('name');
$table->string('email');
$table->integer('age');
$table->timestamps();
});
```

Run migration

Bash

```bash
php artisan migrate
```

9 — Insert Data (Create)

Eloquent method

PHP

```php
Student::create([
'name'=>'John',
'email'=>'john@gmail.com',
'age'=>20
]);
```

Explanation

`create()` inserts a new record into the database.

Equivalent SQL

SQL

```sql
INSERT INTO students VALUES(...)
```

10 — Retrieve Data (Read)

Example

```php
$students = Student::all();
```

Explanation

Method	Description
all()	Get all records
find(id)	Find specific record

Example

```php
Student::find(1);
```

11 — Update Data

Example

```php
$student = Student::find(1);

$student->name = "David";
$student->save();
```

Explanation

`save()` updates the record in the database.

12 — Delete Data

Example

```php
Student::destroy(1);
```

Explanation

Deletes the record with ID = 1.

Another method:

```php
$student->delete();
```

13 — CRUD Operations Summary

Operation	Eloquent
Create	create()
Read	all(), find()
Update	save()
Delete	destroy()

Explanation

These methods help developers **avoid writing SQL queries**.

14 — Eloquent Relationships

Eloquent supports relationships such as:

- One to One
- One to Many
- Many to Many

Example

```
Course → Students
```

15 — One to Many Relationship

Example

```
Course has many Students
Student belongs to Course
```

Diagram

```
Course
  |
  | hasMany
  ↓
Students
```

16 — Course Model

```
class Course extends Model
{

public function students()
{
return $this->hasMany(Student::class);
}

}
```

Explanation

This defines the relationship between **Course and Student**.

17 — Student Model

```
class Student extends Model
{

public function course()
{
return $this->belongsTo(Course::class);
}

}
```

Explanation

Each **student belongs to one course**.

18 — Using Relationships

Example

‹/› PHP

```php
$courses = Course::with('students')->get();
```

Explanation

This loads **courses with their students**.

19 — Example Output

```
Course: Web Development

Students
- Alice
- Bob
- John
```

Explanation

Eloquent automatically **links related data**.

20 — Summary

In this chapter students learned:

- What **Eloquent ORM** is
- How to **create models**
- How to perform **CRUD operations**
- How to use **database relationships**

Benefits of Eloquent

✔ Easy database interaction
✔ Clean code
✔ Powerful relationships

🧪 2 PRACTICES

Practice 1: Basic CRUD

Task:

- Create `Student` model
- Insert 3 students
- Display all students

Learning Outcome:

Understand basic CRUD using Eloquent.

Practice 2: Update & Delete

Task:

- Update student age
- Delete one student

Learning Outcome:

Practice update & delete operations.

PRACTICES explanation

Step 1: Create Model and Migration

Bash

```bash
php artisan make:model Student -m
```

This creates:

```
app/Models/Student.php
database/migrations/create_students_table.php
```

Step 2: Edit Migration

Path:

```
database/migrations/create_students_table.php
```

PHP

```php
public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->integer('age');
        $table->timestamps();
    });
}
```

Run migration

Bash

```bash
php artisan migrate
```

Explanation

This creates a **students table** in MySQL.

PRACTICES explanation

Step 3: Student Model

Path:

```
app/Models/Student.php
```

PHP

```php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    protected $fillable = [
        'name',
        'email',
        'age'
    ];
}
```

Explanation

`$fillable` allows Laravel to **insert data using mass assignment**.

Step 4: Create Controller

Bash

```bash
php artisan make:controller StudentController
```

Path:

```
app/Http/Controllers/StudentController.php
```

Insert and display students:

Explanation

Method	Function
create()	Insert data
all()	Retrieve all records

```php
use App\Models\Student;

class StudentController extends Controller
{

    public function addStudent()
    {
        Student::create([
            'name' => 'David',
            'email' => 'david@gmail.com',
            'age' => 21
        ]);

        return "Student Added";
    }

    public function showStudents()
    {
        $students = Student::all();

        return view('students', compact('students'));
    }

}
```

PRACTICES explanation

Step 5: Route

Path:

```
routes/web.php
```

PHP

```php
use App\Http\Controllers\StudentController;

Route::get('/add-student',[StudentController::class,'addStudent']);
Route::get('/students',[StudentController::class,'showStudents']);
```

Step 6: View

Path:

```
resources/views/students.blade.php
```

HTML

```html
<h2>Student List</h2>

@foreach($students as $s)

<p>
Name: {{$s->name}} <br>
Email: {{$s->email}} <br>
Age: {{$s->age}}
</p>

@endforeach
```

Result

```
localhost:8000/add-student
```

Insert data

```
localhost:8000/students
```

Display students

PRACTICE 2 Update and Delete Student explanation

Goal

Students learn how to **Update and Delete using Eloquent ORM**

Controller Code

Path:

```
app/Http/Controllers/StudentController.php
```

```php
public function updateStudent($id)
{
    $student = Student::find($id);

    $student->name = "Updated Student";
    $student->save();

    return "Student Updated";
}

public function deleteStudent($id)
{
    Student::destroy($id);

    return "Student Deleted";
}
```

Example:

```
localhost:8000/update-student/1
```

Update student ID = 1

```
localhost:8000/delete-student/1
```

Delete student ID = 1

Routes

```
routes/web.php
```

PHP

```php
Route::get('/update-student/{id}',[StudentController::class,'updateStudent']);

Route::get('/delete-student/{id}',[StudentController::class,'deleteStudent']);
```

Explanation

Method	Purpose
find(id)	Find record
save()	Update record
destroy(id)	Delete record

📝 2 ASSIGNMENTS: Product Management System

Task

Create a **Product CRUD system using Laravel Eloquent ORM**

Students must implement:

- Add product
- Display product list
- Update product
- Delete product

Database Table

products

id	name	price	quantity

Step 1: Create Model

Bash

```bash
php artisan make:model Product -m
```

📝 2 ASSIGNMENTS: Management System (Laravel + MySQL + Eloquent ORM)

Project Overview

This project allows you to:

- Add new students
- View all students
- Update student details
- Delete students
- Work with MySQL using **Eloquent ORM**

1️⃣ Project Structure

```
student-management/
├── app/
│   └── Models/
│       └── Student.php
├── app/Http/Controllers/
│       └── StudentController.php
├── database/
│   └── migrations/
│       └── xxxx_create_students_table.php
├── resources/views/
│       ├── students.blade.php
│       ├── create.blade.php
│       └── edit.blade.php
├── routes/
│       └── web.php
└── .env
```

Explanation:

- `Models` → Represents database tables
- `Controllers` → Handles logic (CRUD)
- `Views` → Display HTML pages
- `Migrations` → Create database tables
- `Routes` → Web URLs

📝 2 ASSIGNMENTS: Management System (Laravel + MySQL + Eloquent ORM)

2️⃣ Database Configuration

Open `.env` file:

```env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=student_db
DB_USERNAME=root
DB_PASSWORD=
```

Create the database in MySQL:

```SQL
CREATE DATABASE student_db;
```

3️⃣ Create Student Model + Migration

Run command:

```Bash
php artisan make:model Student -m
```

Migration file (`database/migrations/xxxx_create_students_table.php`):

```PHP
public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->integer('age');
        $table->timestamps();
    });
}
```

Run migration:

```Bash
php artisan migrate
```

📝 2 ASSIGNMENTS: Management System (Laravel + MySQL + Eloquent ORM)

4️⃣ Student Model

Path: `app/Models/Student.php`

‹/› PHP

```php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    protected $fillable = ['name', 'email', 'age'];
}
```

Explanation:

`$fillable` allows mass assignment when using `Student::create()`.

📝 2 ASSIGNMENTS: Management System (Laravel + MySQL + Eloquent ORM)

5 Student Controller

Run:

Bash

```bash
php artisan make:controller StudentController
```

Path: `app/Http/Controllers/StudentController.php`

PHP

```php
namespace App\Http\Controllers;

use App\Models\Student;
use Illuminate\Http\Request;

class StudentController extends Controller
{
    // Show all students
    public function index()
    {
        $students = Student::all();
        return view('students', compact('students'));
    }

    // Show form to create student
    public function create()
    {
        return view('create');
    }

    // Store new student
    public function store(Request $request)
    {
        Student::create($request->all());
        return redirect()->route('students.index')
                        ->with('success','Student created successfully
    }

    // Show form to edit student
    public function edit(Student $student)
    {
        return view('edit', compact('student'));
    }

    // Update student
    public function update(Request $request, Student $student)
    {
        $student->update($request->all());
        return redirect()->route('students.index')
                        ->with('success','Student updated successfully
    }

    // Delete student
    public function destroy(Student $student)
    {
        $student->delete();
        return redirect()->route('students.index')
                        ->with('success','Student deleted successfully
    }
}
```

Explanation:

- `index()` → Display all students
- `create()` → Show form
- `store()` → Save new student
- `edit()` → Show edit form
- `update()` → Update student
- `destroy()` → Delete student

📝 2 ASSIGNMENTS: Management System (Laravel + MySQL + Eloquent ORM)

6️⃣ Routes

Path: `routes/web.php`

‹/› PHP

```php
use App\Http\Controllers\StudentController;

Route::get('/', function(){ return redirect()->route('students.index'); });

Route::resource('students', StudentController::class);
```

Explanation:

`Route::resource` automatically creates **all CRUD routes**.

📝 2 ASSIGNMENTS: Management System (Laravel + MySQL + Eloquent ORM)

7 Views

7.1 students.blade.php (List Students)

Path: `resources/views/students.blade.php`

HTML

```html
<h2>Student List</h2>

<a href="{{ route('students.create') }}">Add Student</a>

@if(session('success'))
    <p style="color:green;">{{ session('success') }}</p>
@endif

<table border="1" cellpadding="5">
    <tr>
        <th>ID</th><th>Name</th><th>Email</th><th>Age</th><th>Actions</th>
    </tr>
    @foreach($students as $student)
    <tr>
        <td>{{ $student->id }}</td>
        <td>{{ $student->name }}</td>
        <td>{{ $student->email }}</td>
        <td>{{ $student->age }}</td>
        <td>
```

```html
<a href="{{ route('students.edit', $student->id) }}">Edit</a> |
<form action="{{ route('students.destroy', $student->id) }}" method="POST"
style="display:inline;">
@csrf
@method('DELETE')
<button type="submit" onclick="return confirm('Delete student?')">Delete</button>
</form>
</td>
</tr>
@endforeach
</table>
```

📝 2 ASSIGNMENTS: Management System (Laravel + MySQL + Eloquent ORM)

7.2 create.blade.php (Add Student Form)

Path: `resources/views/create.blade.php`

HTML

```html
<h2>Add Student</h2>

<form action="{{ route('students.store') }}" method="POST">
    @csrf
    Name: <input type="text" name="name" required><br><br>
    Email: <input type="email" name="email" required><br><br>
    Age: <input type="number" name="age" required><br><br>
    <button type="submit">Add Student</button>
</form>

<a href="{{ route('students.index') }}">Back</a>
```

7.3 edit.blade.php (Edit Student Form)

Path: `resources/views/edit.blade.php`

HTML

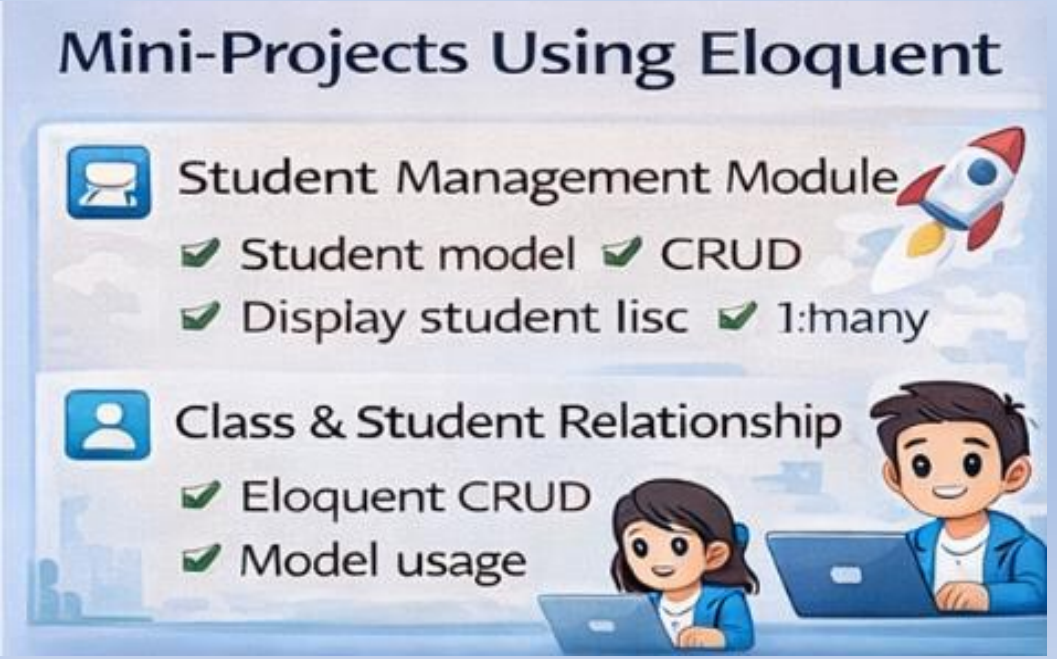
```html
<h2>Edit Student</h2>

<form action="{{ route('students.update', $student->id) }}" method="POST">
    @csrf
    @method('PUT')
    Name: <input type="text" name="name" value="{{ $student->name }}" required><br><b
    Email: <input type="email" name="email" value="{{ $student->email }}" required><b
    Age: <input type="number" name="age" value="{{ $student->age }}" required><br><br
    <button type="submit">Update Student</button>
</form>

<a href="{{ route('students.index') }}">Back</a>
```

🧩 2 MINI-PROJECTS

🚀 **Mini-Project 2: Class & Student Relationship**

Features:

- Models:
  - Class
  - Student
- One-to-Many relationship
- Show students under a class

Skills Gained:

- Relationships
- Real-world database linking